ASTANA IT UNIVERSITY

School of Software Engineering

Specialty: 6B06102 — Software Engineering

DEVELOPMENT OF A WEB PLATFORM FOR REAL-TIME AUTHORIZED SEARCH OF ACADEMIC SUPERVISORS

By: Damir Bekturov

Tairlan Zhanashev

Balzhan Komekbaeva

Supervisor: Omirgaliyev Ruslan

Senior Lecturer

School of Software Engineering

ASTANA, JUNE 2026

Abstract

SupervisorMatch is a full-stack web application developed to address the inefficiencies inherent in the traditional academic supervisor selection process at Astana IT University. The system provides a centralised, role-aware digital platform through which students may browse supervisor profiles, examine available research topics, and submit formal supervision requests, while supervisors may manage incoming applications, set their capacity limits, and maintain an up-to-date catalogue of research interests.

The application is built upon a modern, industry-standard technology stack: React 18 with Vite and Tailwind CSS on the client side, NestJS with TypeScript and JWT-based authentication on the server side, PostgreSQL as the relational database, and Prisma ORM for type-safe data access. The system is fully containerised using Docker and Docker Compose, and served through an Nginx reverse proxy for production-ready deployment.

A user survey of 24 Astana IT University students confirmed that over 70% found the existing supervisor discovery process difficult or partially opaque, and more than 80% expressed willingness to use a centralised web platform. SupervisorMatch directly targets these pain points by combining real-time workload visibility, keyword-based supervisor search, and a structured online request workflow. The project contributes a replicable architectural pattern for role-based academic management systems applicable to higher-education institutions more broadly.

Keywords: full-stack web application, supervisor matching, NestJS, React, PostgreSQL, Prisma ORM, JWT authentication, role-based access control, Docker, academic information system.

Dedication and Acknowledgements

To our families, whose unwavering support and patience made this journey possible.

The authors would like to express their sincere gratitude to their academic supervisor, *Omirgaliyev Ruslan*, for providing guidance, constructive criticism, and continuous encouragement throughout the development and documentation phases of this project.

We also acknowledge the students of Astana IT University who participated in our survey and whose candid feedback shaped the functional priorities of SupervisorMatch. Their insights were invaluable in grounding this project in real-world need rather than theoretical assumption.

Finally, we thank the open-source communities behind NestJS, React, Prisma, and Docker. Their freely available tools and comprehensive documentation form the technological foundation upon which SupervisorMatch was built.

Author's Declaration

We, **Balzhan Komekbaeva**, **Damir Bekturov**, and **Tairlan Zhanashev**, hereby declare that this project report, entitled *"Development of a Full-Stack Web Application 'SupervisorMatch' for Automated Academic Supervisor Discovery and Request Management"*, submitted for the award of the Bachelor's degree in Software Engineering at Astana IT University, is a record of original work carried out by us under the supervision of *Omirgaliyev Ruslan, Senior Lecturer, School of Software Engineering.*

We further declare that:

1. The work presented in this report is original and has not previously been submitted for any degree or academic award at this or any other institution.
2. All sources of information used in this report have been appropriately cited and referenced in accordance with academic conventions.
3. Any assistance received during the course of this work has been duly acknowledged.
4. All intellectual property rights in this work are assigned to Astana IT University in accordance with the university's intellectual property policy.

Komekbaeva B.: _____ Date: _____

Bekturov D.: _____ Date: _____

Zhanashev T.: _____ Date: _____

Definitions

Supervisor An academic staff member registered in the SupervisorMatch system who has the authority to accept or reject student supervision requests, define research topics, and manage their available capacity.

Capacity The maximum number of students a supervisor is willing to accept for supervision in a given academic cycle. The system enforces this limit automatically, preventing students from submitting requests to fully booked supervisors.

Supervision Request A formal, digitally submitted application from a student to a supervisor expressing interest in a particular research topic and requesting guidance for a thesis or course project.

JWT (JSON Web Token) An open industry standard (RFC 7519) for creating access tokens that securely transmit information between parties as a JSON object. Used in SupervisorMatch for stateless authentication and authorisation.

Prisma ORM An open-source next-generation Object-Relational Mapper for Node.js and TypeScript that provides type-safe database access, auto-generated queries, and schema migration tools. Used to interface with the PostgreSQL database.

NestJS A progressive Node.js framework for building efficient, scalable, and maintainable server-side applications using TypeScript and a module-based architecture inspired by Angular.

React A JavaScript library for building user interfaces based on a component model. SupervisorMatch uses React 18 with the Vite build tool.

Docker An open platform for developing, shipping, and running applications inside lightweight, portable containers, ensuring environment consistency across development and production.

Nginx A high-performance HTTP server and reverse proxy used in SupervisorMatch to route incoming web traffic to the appropriate frontend or backend service.

Role-Based Access Control (RBAC)

An access control paradigm that restricts system operations based on the roles assigned to users. SupervisorMatch defines two roles: `STUDENT` and `SUPERVISOR`.

Designations and Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
SPA	Single-Page Application
ORM	Object-Relational Mapper
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JWT	JSON Web Token
SQL	Structured Query Language
RBAC	Role-Based Access Control
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
CI/CD	Continuous Integration / Continuous Deployment
AITU	Astana IT University
MVC	Model-View-Controller
DBMS	Database Management System

Contents

List of Tables

List of Figures

Introduction

Background and Context

The selection of an appropriate academic supervisor is one of the most consequential decisions a university student undertakes during the preparation of a thesis or course project. A well-matched supervisor not only provides domain-specific expertise but also shapes the student's research methodology, professional development, and academic outcomes. Despite this importance, the process through which students identify, evaluate, and contact potential supervisors at many institutions—including Astana IT University—remains largely informal, fragmented, and opaque.

In the absence of a centralised information system, students typically resort to scanning static departmental web pages, consulting peer networks, or sending unsolicited emails to faculty members whose suitability is assessed through incomplete or outdated profiles. Supervisors, in turn, lack a unified channel through which to communicate their research interests, announce their availability, or respond to requests in an auditable, trackable fashion. The result is a significant coordination inefficiency: students spend disproportionate time in discovery activities, supervisors receive unstructured enquiries, and departmental administrators lack visibility into the distribution of supervision workloads.

Project Objectives

SupervisorMatch was conceived and developed to resolve these structural deficiencies. The primary objective of this project is to design and implement a full-stack web application that digitalises and formalises the supervisor discovery and request management process at Astana IT University. Specific objectives include: (1) enabling students to search and filter supervisor profiles by research keywords, department, and availability; (2) allowing supervisors to define their research topics, set capacity limits, and process incoming requests through a dedicated dashboard; (3) enforcing role-based access control to ensure that students and supervisors access only the functionality appropriate to their roles; and (4) providing a responsive, accessible interface deployable in a containerised production environment.

Scope and Limitations

The scope of SupervisorMatch encompasses the full development lifecycle of the application, from requirements analysis and architectural design through implementation, testing, and

containerised deployment. The system targets the two primary user roles (Student and Supervisor) and includes core modules for authentication, supervisor discovery, topic management, and request processing.

The following limitations are acknowledged: the current version does not include an AI-based recommendation engine, though this is identified as a future enhancement. The system does not integrate with the university's existing student information system or single sign-on infrastructure in this iteration, relying instead on its own JWT-based authentication. Email notifications are outside the current scope, as is a mobile-native application. These limitations are deliberate boundary decisions to maintain project focus and are catalogued as future work in the Conclusion chapter.

Chapter 1. Literature Review

1.1 Academic Supervision and the Student-Supervisor Matching Process

Academic supervision is widely recognised in higher education research as a critical determinant of student success, particularly at the graduate and final-year undergraduate levels. Johnson [1] defines effective supervision as a relational practice in which the supervisor acts simultaneously as mentor, critic, and collaborator, guiding the student through the full arc of research from problem formulation to dissemination. The quality of this relationship is contingent, in large part, on the alignment of intellectual interests, working styles, and mutual expectations between supervisor and student.

The matching problem in academic supervision shares structural characteristics with other two-sided matching markets studied in economics and computer science [2]. Each student seeks a supervisor whose research agenda intersects with their own interests, while each supervisor is constrained by capacity and seeks students whose backgrounds and commitments align with the demands of proposed projects. Traditional manual processes—email campaigns, word-of-mouth referrals, departmental notice boards—produce suboptimal matches because neither party has complete visibility into the preferences and constraints of the other.

Empirical studies in educational technology confirm that digitally mediated matching systems improve both match quality and process efficiency. Park and Lee [3] demonstrate in a Korean university context that students who used an online supervisor recommendation portal reported significantly higher satisfaction with their chosen supervisor than those who relied on manual procedures. These findings corroborate the theoretical expectation that information asymmetry is the primary driver of matching inefficiency and that digital platforms can substantially reduce it. Similarly, Ismail et al. [4] propose a Euclidean-distance recommender engine for final-year project supervisor matching at a Malaysian university, providing an early proof of concept for automated interest-based matching. More recently, Braescu and Davidescu [6] synthesise the state of the art in supervisor selection systems and propose a web application combining a recommendation system with learning management functionality, using matrix factorisation and cosine similarity over research topic embeddings—an approach that SupervisorMatch identifies as a priority for future enhancement.

1.2 Web Application Development for Academic Environments

The development of information systems for academic environments has been an active area of applied software engineering research since the early 2000s. Early systems were predominantly monolithic, server-rendered applications built with technologies such as PHP, JSP, and ASP.NET. Contemporary educational information systems increasingly adopt a decoupled architecture in which a REST or GraphQL API backend serves a single-page application frontend, enabling greater flexibility, mobile compatibility, and independent scalability of components [8].

The prevalence of JavaScript across both browser and server environments has accelerated the adoption of unified JavaScript ecosystems in academic system development. Frameworks such as NestJS on the server and React on the client have emerged as industry standards due to their modular architectures, extensive ecosystem support, and strong TypeScript integration. Piastou [17] provides a comprehensive performance benchmark of React, Angular, and Vue.js under increasing load, finding that React's virtual DOM diffing algorithm delivers superior rendering throughput in data-intensive interfaces—a property directly relevant to SupervisorMatch's supervisor directory page, which renders dynamic lists of profile cards. Jonathan and Suprihadi [16] further validate the React-plus-SPA architectural pattern for academic management systems, demonstrating that role-conditioned routing and component-level authorisation can be implemented cleanly within the React Router ecosystem.

Security requirements in academic environments add additional constraints to web application design. Systems handling student personal data are subject to data protection regulations, and any authentication mechanism must resist common web vulnerabilities, including cross-site scripting, cross-site request forgery, and brute-force attacks on credentials. The adoption of stateless JWT-based authentication, validated against a server-side secret, satisfies these requirements while avoiding the scalability limitations of server-side session storage [10]. Bhatti et al. [14] further note that enforcement of access control at the API level is a necessary but insufficient condition for usability—client-side enforcement is equally required to prevent users from attempting operations that the backend will silently reject, a principle reflected in SupervisorMatch's role-conditional UI rendering.

1.3 Role-Based Access Control in Educational Systems

Role-Based Access Control (RBAC) is a well-established security paradigm first formalised by Ferraiolo and Kuhn [11] and subsequently standardised by NIST [12]. The model assigns

permissions to roles rather than to individual users, and grants users one or more roles, thereby separating the concerns of permission management and user administration. In educational information systems, RBAC enables the definition of distinct access boundaries between students, instructors, supervisors, and administrative staff.

The implementation of RBAC in modern web applications typically takes one of two forms: coarse-grained role checking at the API gateway level, or fine-grained attribute-based policies applied at the resource level. SupervisorMatch adopts the coarse-grained approach, applying role guards at the NestJS controller level via custom decorators. This approach is appropriate given the relatively small number of distinct roles (Student and Supervisor) and the clarity of the access boundaries between them. Shin et al. [15] propose a modelling and testing methodology for dynamic RBAC in web applications, highlighting that role and permission assignments frequently evolve as systems mature—a finding that informs SupervisorMatch's decision to centralise role definitions in a Prisma enum rather than hardcoding them in controller logic.

Research on RBAC in academic systems highlights the importance of enforcing role constraints at both the API and UI levels. A backend-only enforcement strategy, while secure, provides a poor user experience because users may attempt actions that are silently rejected. SupervisorMatch addresses this by conditionally rendering role-specific navigation items and action buttons on the frontend, guided by the decoded JWT role claim, thereby ensuring that the user interface faithfully reflects the permissions of the authenticated user [13].

1.4 Existing Supervisor Matching Systems

Several institutions have developed proprietary or semi-proprietary systems for supervisor assignment. The University of Queensland's postgraduate research candidature system, for example, provides a searchable directory of supervisor profiles linked to the university's research output repository. Similarly, the UK Research and Innovation portal enables prospective doctoral students to filter funded projects by supervisor and institution. These systems, while functional, are typically tightly coupled to institutional identity management infrastructure and are not designed for adoption at smaller or newly established universities [**?**].

Commercial platforms such as FindASupervisor.com and ProFellow.com offer cross-institutional matching at the postgraduate level but operate on an opt-in model that limits data completeness and currency. None of these platforms provides a request management workflow that allows supervisors to formally accept or decline applications within the platform itself; the communication is invariably redirected to external email. This absence of an integrated request lifecycle is a significant gap that SupervisorMatch is designed to fill.

Open-source alternatives are scarce. The Moodle Learning Management System, while widely deployed in academic contexts, does not include a supervisor matching module. Some institutions have extended Moodle with custom plugins to facilitate project allocation, but these solutions are highly context-specific and lack the profile-centric supervisor discovery interface that students require for informed selection.

1.5 Comparative Analysis of Existing Solutions

Table 1.1 presents a structured comparison of SupervisorMatch against four representative existing systems across six capability dimensions.

Table 1.1: Comparative analysis of supervisor matching platforms

Feature	**SupervisorMatch**	**UQ System**	**UKRI Portal**	**FindASupervisor**	**Moodle+Plugin**
Keyword Search					○
Capacity Visibility		×	×	×	○
Online Requests		×	×	×	
RBAC			○	×	
Containerised		×	×	×	×
Open Source		×	×	×	

= Fully supported; ○ = Partially supported; × = Not supported

The analysis reveals that SupervisorMatch is the only system in the comparison set to simultaneously offer capacity visibility and an integrated online request workflow. The absence of capacity information in competing systems is a notable shortcoming: students have no way of determining which supervisors are available before initiating contact, leading to wasted effort when preferred supervisors are fully booked. SupervisorMatch addresses this by surfacing real-time capacity data on every supervisor profile card.

The fully containerised deployment model is another differentiating feature. Competing systems are largely hosted on institutional infrastructure managed by central IT departments, making them difficult to adapt or redeploy. SupervisorMatch's Docker-based deployment allows any institution to spin up a functional instance with a single `docker-compose up` command, significantly lowering the barrier to adoption. Sharma [22] demonstrates that Docker containerisation reduces environment-related deployment failures by eliminating dependency conflicts between development and production environments, while Cherukuri [24] provides a comparative analysis confirming Docker Compose's suitability for single-host multi-service orchestration in applications of SupervisorMatch's scale.

1.6 Survey Results Analysis

In order to ground the development of SupervisorMatch in empirically observed student experience, a structured online survey was administered to students of Astana IT University in January 2026. A total of 24 valid responses were collected, predominantly from Bachelor-level students (20 respondents, 83.3%) with four Master-level participants (16.7%). The survey instrument

comprised thirteen items covering study level, current supervisor requirements, search methods, perceived difficulty, information availability, number of contacts made, experienced difficulties, time spent, perceived transparency, willingness to use a centralised platform, desired features, and importance of institutional authorisation. Selected open-text responses were also collected.

Survey Instrument and Sample Profile

Of the 60 respondents, 40 (66.7%) confirmed that they are currently required to choose a supervisor for a course project or thesis, while the remaining 20 were either not yet required or uncertain about their status. The predominant supervisor discovery methods employed were departmental website or lists (used by 65% of respondents) and peer consultation (asking classmates or senior students, used by 45%), with direct email contact used by 40% of participants. Only 3 respondents reported consulting an academic coordinator directly, underscoring the absence of a formalised institutional channel.

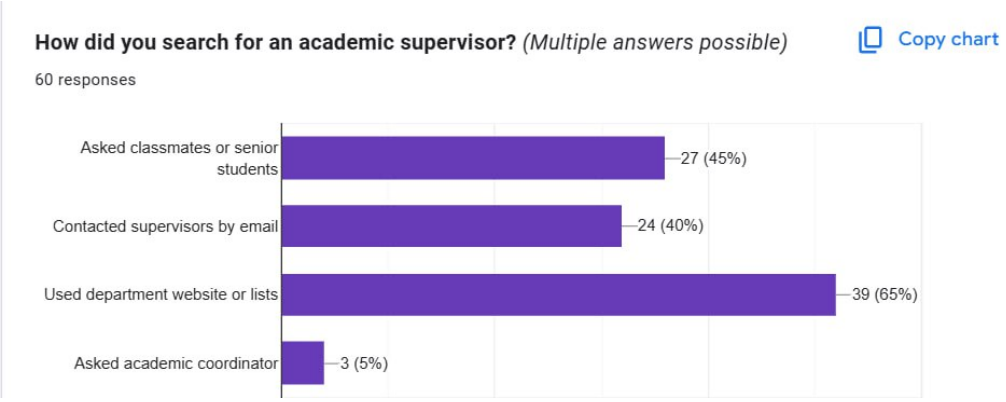

Figure 1.1: Methods used by AITU students to search for academic supervisors ($n = 60$)

Perceived Difficulty and Information Availability

Respondents rated the ease of finding information about supervisors' research interests on a five-point Likert scale (1 = Very Difficult, 5 = Very Easy). The majority of respondents selected the neutral option, with 33 out of 60 respondents (55.0%) choosing rating 3. Ratings of 1 (Very Difficult) and 2 (Difficult) were each selected by 6 respondents (10.0%), resulting in a combined difficulty rate of 20.0% for responses below the midpoint. In contrast, 12 respondents (20.0%) rated the experience as 4 (Easy), while only 3 respondents (5.0%) selected 5 (Very Easy). These findings suggest that although most students did not perceive the process as extremely difficult, accessing information about supervisors' research interests was not considered particularly straightforward either.

Supervisor workload visibility emerged as a significant information gap. Only 15 out of 60 respondents (25.0%) reported that information about supervisors' current supervision workload was clearly available. The majority, 42 respondents (70.0%), stated that such information was only partially available, while 3 respondents (5.0%) indicated that it was not available at all. These results demonstrate that for approximately three-quarters of students, determining a

supervisor's current availability remains a challenging and insufficiently supported process within existing information channels.

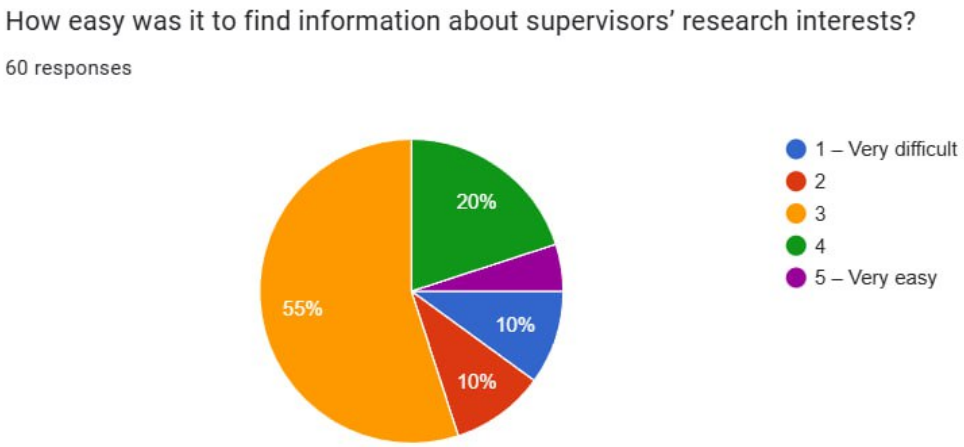

Figure 1.2: Availability of supervisor workload information as reported by survey respondents

Number of Contacts and Process Duration

Students were asked how many supervisors they had contacted before receiving approval. The distribution is presented in Table 1.2. A considerable proportion of respondents needed to contact multiple supervisors before securing approval. In particular, 20 out of 60 respondents (33.3%) reported contacting three or more supervisors, while 5 respondents (8.3%) had not yet received approval at the time of survey completion. The largest group, 27 respondents (45.0%), received approval after contacting two supervisors, whereas only 8 respondents (13.3%) were successful after contacting a single supervisor. These findings suggest that students often experience repeated attempts and delays during the supervisor selection process, largely due to limited transparency regarding supervisor availability and research compatibility.

Table 1.2: Number of supervisors contacted before approval

Number Contacted	Frequency	Percentage
One	8	13.3%
Two	27	45%
Three or more	20	33.3%
Not yet approved	5	8.3%

Regarding process duration, a substantial proportion of students required more than a short period to complete the supervisor selection process. The findings indicate that delays in communication and uncertainty about supervisor availability can significantly extend the time needed to secure approval. As a result, many students begin their research activities later than planned, reducing the effective time available for project development and preparation.

Experienced Difficulties and Perceived Transparency

The most commonly reported difficulty was supervisor overload or lack of availability, cited by 30 respondents (50.0%). This was followed by slow response or no response from supervisors, reported by 27 respondents (45.0%), and mismatch of research interests, selected by 23 respondents (38.3%). In addition, 18 respondents (30.0%) identified the lack of transparent information as a major issue, while 3 respondents (5.0%) mentioned that the approval process itself was overly complicated.

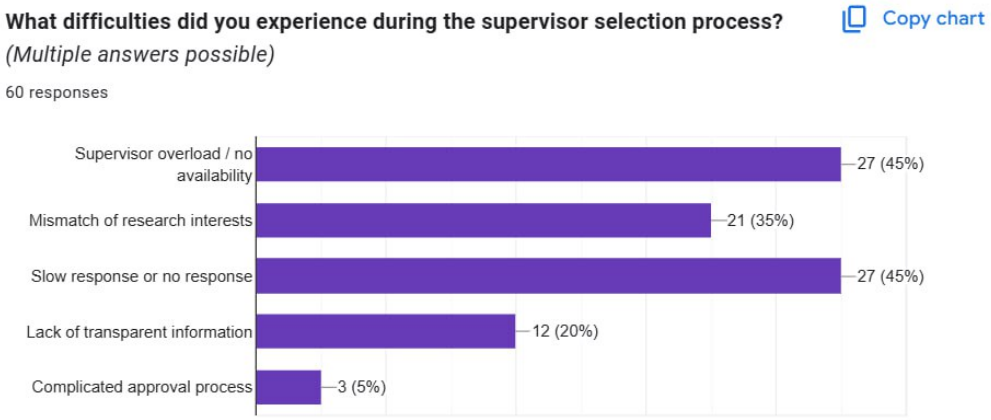

Figure 1.3: Difficulties experienced by AITU students during the supervisor selection process

Overall process transparency was evaluated using a five-point scale (1 = Not Transparent at All, 5 = Fully Transparent). The mean rating remained approximately 3.0, indicating a moderate level of perceived transparency. Ratings of 1 and 2 were each selected by approximately 10 respondents (16.7%), while the majority of respondents, 27 students (45.0%), chose rating 3. Higher ratings of 4 or 5 were given by around 18 respondents (30.0%). The dominance of the neutral midpoint suggests that many students perceive the current supervisor selection process as ambiguous rather than clearly transparent or entirely opaque. This finding highlights the need for a more structured and centralized information system to support supervisor discovery and selection.

Chapter 2. Design and Architecture

2.1 Methodology and Development Approach

SupervisorMatch was developed using an Agile methodology with two-week Sprint cycles inspired by the Scrum framework. The Agile approach was selected for its iterative nature, which allowed the team to continuously incorporate feedback from preliminary user testing and to reprioritise the product backlog in response to emerging requirements. The Agile Manifesto [26] identifies "responding to change over following a plan" as a core value; this principle proved especially pertinent during Sprint 3, when user testing revealed that the initial request submission UI design created ambiguity around capacity status. The Sprint Review ceremony allowed the team to surface this finding and reprioritise the capacity badge feature for immediate implementation. Pressman and Maxim [27] advocate the MoSCoW prioritisation technique within Agile backlogs as a mechanism for ensuring that contractual "Must-Have" features are delivered before "Could-Have" enhancements, a practice adopted throughout the SupervisorMatch development process.

The development process was structured across five Sprints. Sprint 1 focused on requirements gathering, technology selection, and project scaffolding. Sprints 2 and 3 addressed core backend functionality: authentication, user management, supervisor profiles, topics, and the request workflow. Sprint 4 developed the React frontend, including the public supervisor discovery interface and the authenticated dashboards for both roles. Sprint 5 was dedicated to integration testing, Docker containerisation, deployment configuration, and documentation.

User stories were maintained in a shared backlog and categorised by priority using the MoSCoW method (Must-Have, Should-Have, Could-Have, Won't-Have). This ensured that the core supervision request workflow was delivered and tested before effort was invested in secondary features such as profile image upload or keyword tagging.

2.2 Requirements Analysis

Functional Requirements

FR1. The system shall allow users to register as either a Student or a Supervisor.

FR2. The system shall authenticate users using email and password credentials and issue a JWT access token upon successful login.

FR3. The system shall allow Students to browse the list of all registered Supervisors and view individual Supervisor profiles.

FR4. The system shall display each Supervisor's research interests, available topics, and remaining capacity.

FR5. The system shall allow Students to submit a supervision request to any Supervisor with available capacity.

FR6. The system shall allow Supervisors to view all incoming requests and approve or reject them.

FR7. The system shall enforce the Supervisor's capacity limit: no new requests shall be accepted once the limit is reached.

FR8. The system shall allow Supervisors to create, edit, and delete research topics on their profile.

FR9. The system shall restrict dashboard access based on user role (Student dashboard / Supervisor dashboard).

FR10. The system shall allow Supervisors to update their profile information, including bio, department, and capacity.

Non-Functional Requirements

NFR1. **Security:** All API endpoints requiring authentication shall validate the JWT bearer token. Passwords shall be hashed using bcrypt.

NFR2. **Performance:** The API shall respond to standard data retrieval requests within 500ms under normal load.

NFR3. **Usability:** The user interface shall be responsive and accessible on desktop and mobile screen widths from 375px upward.

NFR4. **Maintainability:** The codebase shall follow NestJS module conventions and React component best practices, with a separation between business logic and presentation layers.

NFR5. **Portability:** The entire application stack shall be deployable using a single `docker-compose up` command.

NFR6. **Reliability:** The system shall use database transactions to ensure request state consistency during concurrent submissions.

2.3 System Architecture

2.3.1 Architectural Style and Justification

SupervisorMatch adopts a client-server architectural style with a RESTful API communication layer. The frontend Single-Page Application (SPA) communicates with the backend exclusively through HTTP requests to versioned REST endpoints (`/api/v1/...`). This decoupled architecture provides several advantages: the frontend and backend can be developed and tested independently, the API can be consumed by future clients (e.g., a mobile application) without modification, and the backend can be scaled horizontally behind a load balancer without changes to the client.

The REST paradigm was preferred over GraphQL on the grounds of implementation simplicity and the well-understood tooling ecosystem around NestJS REST controllers. Given the relatively small and well-defined entity model (User, Supervisor, Topic, Request), the flexible querying capabilities of GraphQL were not required.

2.3.2 Context Diagram

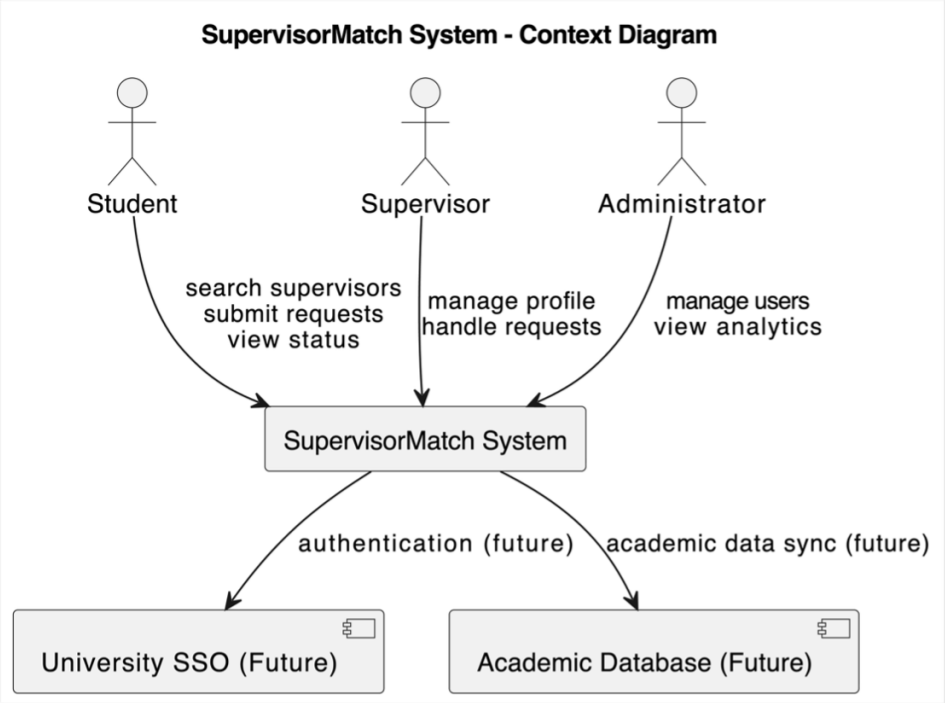

Figure 2.1: System context diagram for SupervisorMatch

The system context diagram (Figure 2.1) situates SupervisorMatch within its operating environment by showing the system as a single black-box entity surrounded by four external actors. The **Student** actor interacts with the system via a web browser, performing operations such as browsing the supervisor catalogue, viewing individual profiles, and submitting supervision requests. The **Supervisor** actor similarly accesses the system through the browser to manage their profile, publish research topics, and respond to incoming student requests. The **System Administrator** actor, whose role is reserved for future releases, represents institutional staff who would manage user accounts and monitor workload distribution. Finally, the **PostgreSQL Database** is shown as an external data store with which the system exchanges persistent data. All interactions between actors and the system cross the system boundary as HTTPS requests routed through the Nginx reverse proxy; no actor communicates directly with internal components such as the NestJS API or the database.

2.3.3 Component Diagram

The component diagram (Figure 2.2) decomposes the SupervisorMatch system into its principal deployable units and illustrates the interfaces through which they communicate. The outermost layer is the **Nginx Reverse Proxy**, which receives all inbound HTTP traffic on port 80. For requests whose path begins with `/api/`, Nginx forwards the request upstream to the **NestJS Backend** container on port 3000. All other paths are resolved by Nginx itself by serving the pre-built static files of the **React SPA**.

Within the NestJS backend, four feature modules are shown as internal components: the **Auth Module** (exposes `/auth/register` and `/auth/login`, issues JWTs via the `JwtService`), the

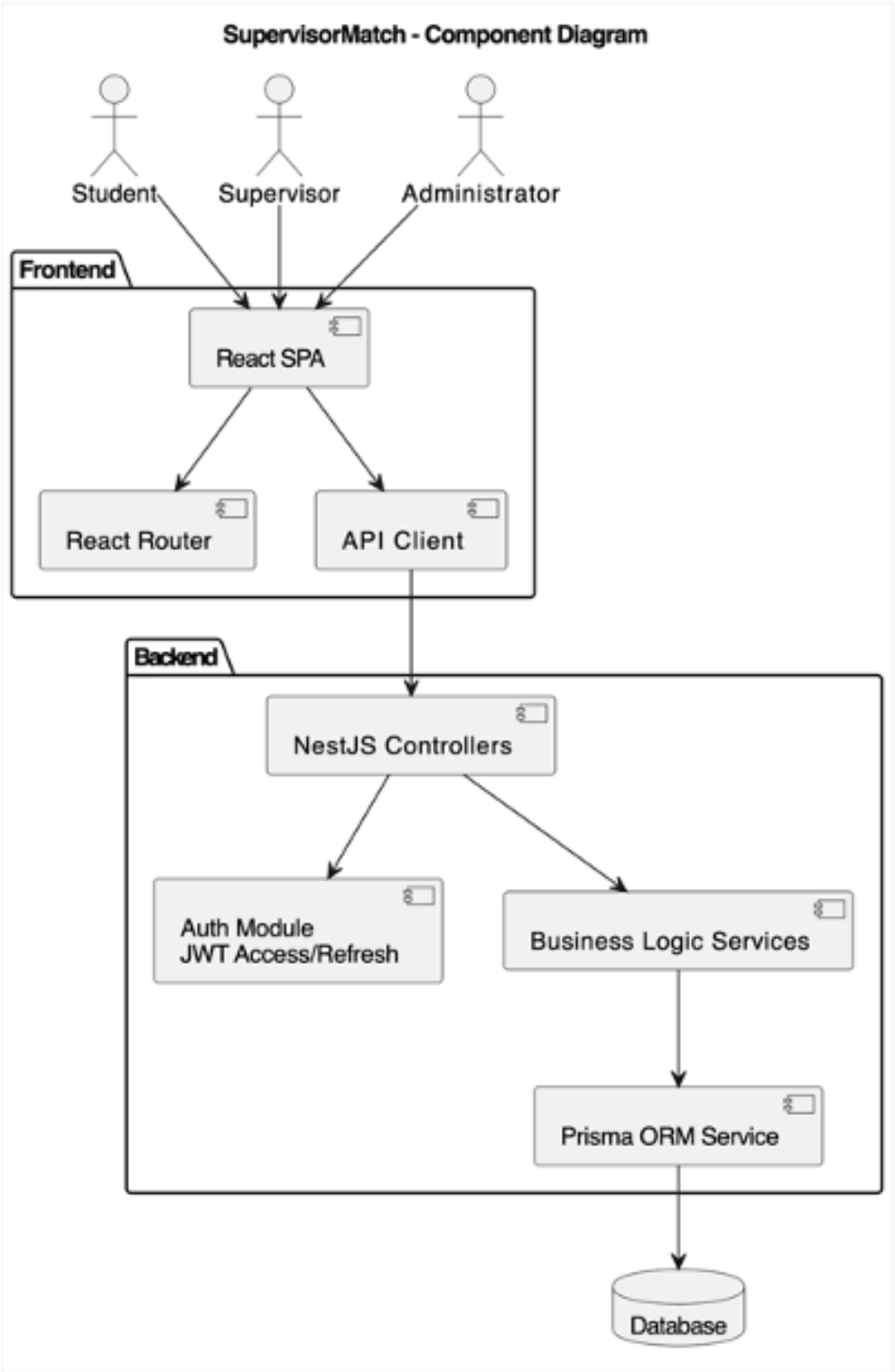

Figure 2.2: Component diagram of the SupervisorMatch system

Supervisors Module (exposes `/supervisors` endpoints, provides profile retrieval and update operations), the **Topics Module** (exposes `/topics` endpoints, handles CRUD for research topics owned by a supervisor), and the **Requests Module** (exposes `/requests` endpoints, enforces capacity constraints and processes approvals). All four modules share a single **Prisma Service** component, which maintains the connection pool to the **PostgreSQL 15** container and translates typed query calls into SQL statements. The React SPA communicates with the NestJS backend exclusively through the `/api/` prefix, carrying a Bearer JWT in the `Authorization` header for all authenticated requests.

2.4 UML Diagrams

2.4.1 Use Case Diagram

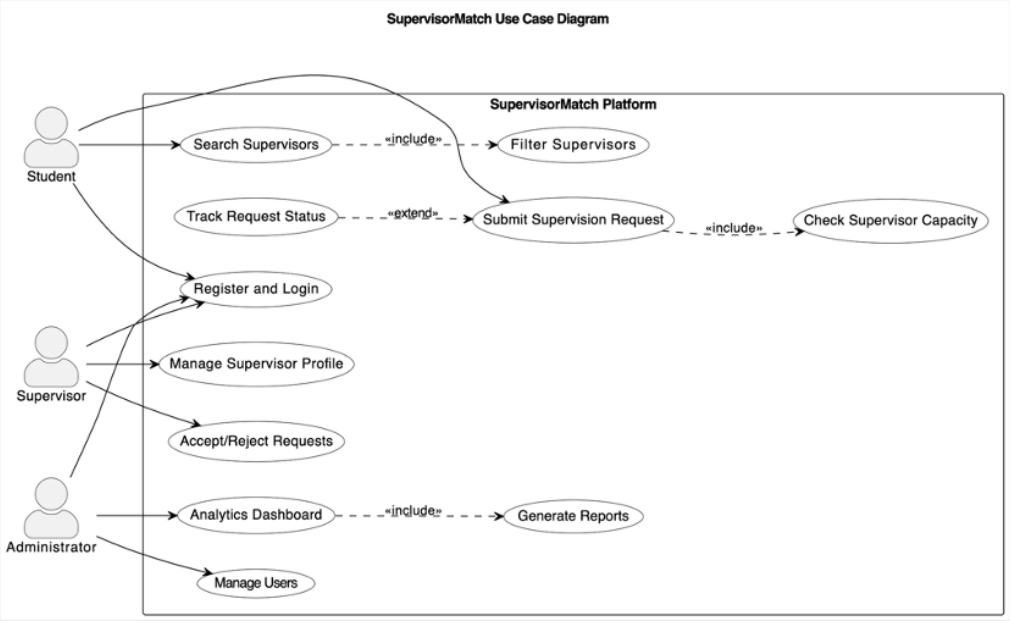

Figure 2.3: Use case diagram for the SupervisorMatch system

The use case diagram (Figure 2.3) identifies two primary actors—**Student** and **Supervisor**—and one secondary actor, **System** (representing automated system-side logic). Both actors share the generalised use cases *Register* and *Login*, which are prerequisites for all role-specific functionality. The *Register* use case is extended by *Select Role*, reflecting the registration flow in which the user designates themselves as either a Student or a Supervisor.

The Student actor is associated with the following use cases: *Browse Supervisor Directory* (viewing the paginated list of supervisor profiles); *Search Supervisors by Keyword* (an extension of Browse, triggered when the student enters a search term); *View Supervisor Profile* (drilling into a specific supervisor's bio, topics, and capacity); and *Submit Supervision Request* (which includes *Check Capacity*, an automated sub-use case invoked by the System to validate available slots before the request record is created). Students can also *Track Request Status*, viewing the current state (Pending, Approved, or Rejected) of all their submitted requests.

The Supervisor actor is associated with: *Manage Profile* (updating bio, department, and capacity limit); *Manage Research Topics* (adding, editing, and deleting topic entries on the profile); *View Incoming Requests* (listing all student applications addressed to the supervisor); *Approve Request* (setting status to Approved and decrementing remaining capacity via the System); and *Reject Request* (setting status to Rejected without affecting capacity). The two approval/rejection use cases include the System-side use case *Update Remaining Capacity*, shown as an included relationship to make the automated capacity management logic explicit.

2.4.2 Sequence Diagrams

Supervisor Search Workflow

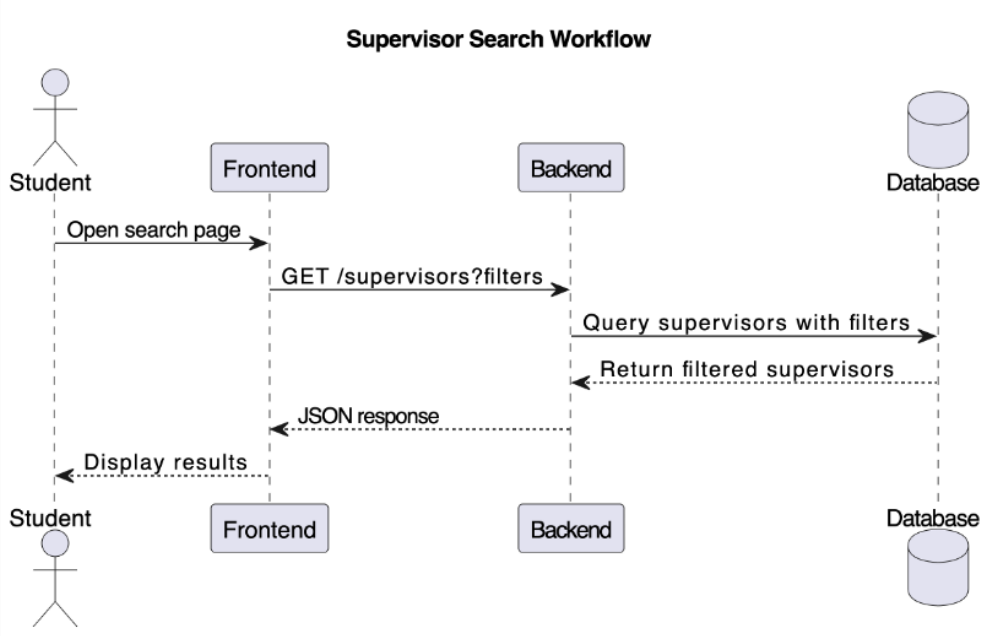

Figure 2.4: Sequence diagram: Supervisor search workflow

The sequence diagram in Figure 2.4 traces the message flow for the supervisor search workflow, which is accessible without authentication. The interaction begins when the **Student** types a keyword (e.g., "machine learning") into the search bar on the Supervisor Directory page rendered by the **React SPA**. The SPA's `SupervisorList` component reacts to the `onChange` event and invokes the `api.getSupervisors(keyword)` function in the `services/api.ts` module, which dispatches an HTTP `GET /api/supervisors?search=machine+learning` request to the **Nginx Proxy**.

Nginx forwards the request to the **NestJS** `SupervisorsController`, which passes the query parameter to the `SupervisorsService`. The service constructs a Prisma `findMany` query with a `where` clause that applies a case-insensitive `contains` filter on both the supervisor's `bio` field and the `title` field of their associated `Topic` records. The **Prisma Client** translates this into a parameterised SQL `SELECT` with an `ILIKE` predicate issued to the **PostgreSQL** instance. The database returns a result set of matching supervisor rows, which Prisma maps to typed TypeScript objects and returns up the call chain. The `SupervisorsController` serialises the list to JSON

and responds with HTTP 200. The SPA receives the response, updates the `supervisors` React state, and re-renders the card grid to display only supervisors whose profile or topics match the search keyword, along with each supervisor's current `remainingCapacity` badge.

Supervision Request Submission Workflow

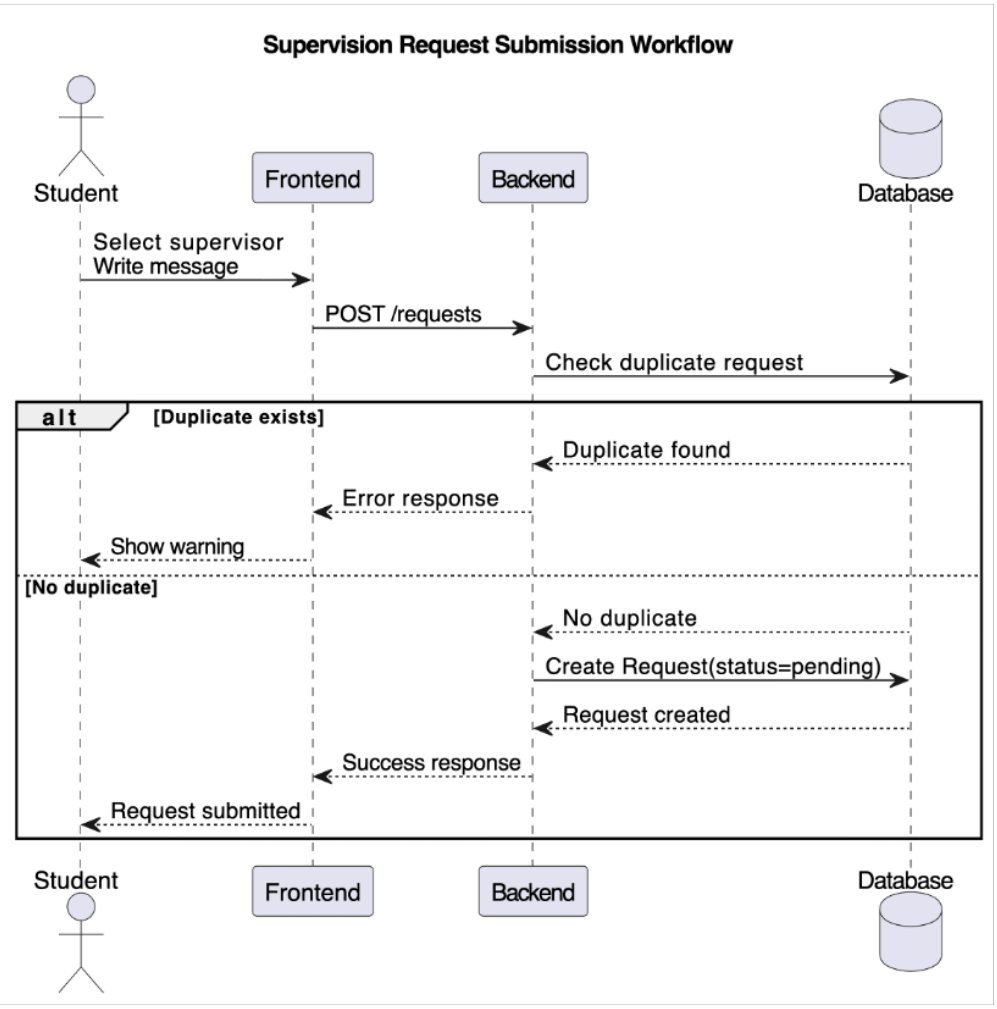

Figure 2.5: Sequence diagram: Supervision request submission workflow

Figure 2.5 details the message exchange that occurs when an authenticated student submits a supervision request. The flow is triggered when the **Student** clicks the "Request Supervision" button on a supervisor's profile page within the **React SPA**. The `RequestForm` component collects an optional `message` and the target `supervisorId` from the page context, then calls `api.submitRequest({supervisorId, message})`. The Axios interceptor automatically attaches the student's stored JWT as a `Bearer` token in the `Authorization` header, and the SPA dispatches `POST /api/requests` to the **Nginx Proxy**, which forwards it to the **NestJS** `RequestsController`.

The controller first invokes the global `JwtAuthGuard`, which extracts and verifies the JWT signature using the server's secret. If the token is valid, the guard attaches the decoded user payload (including `userId` and `role`) to the request object and allows execution to continue. The controller then delegates to `RequestsService.createRequest()`, which calls **Prisma** to retrieve

the target supervisor's current `remainingCapacity` from **PostgreSQL**. If `remainingCapacity` is zero, the service throws a `400 Bad Request` exception and the flow terminates; the SPA displays an "This supervisor has no available slots" error message to the student. If capacity is available, the service creates a new `Request` record with `status: PENDING` via a Prisma `create` call, PostgreSQL confirms the insert, and the API returns `HTTP 201 Created` with the new request object. The SPA navigates the student to their dashboard, where the newly submitted request appears with a "Pending" status badge.

Request Approval Workflow

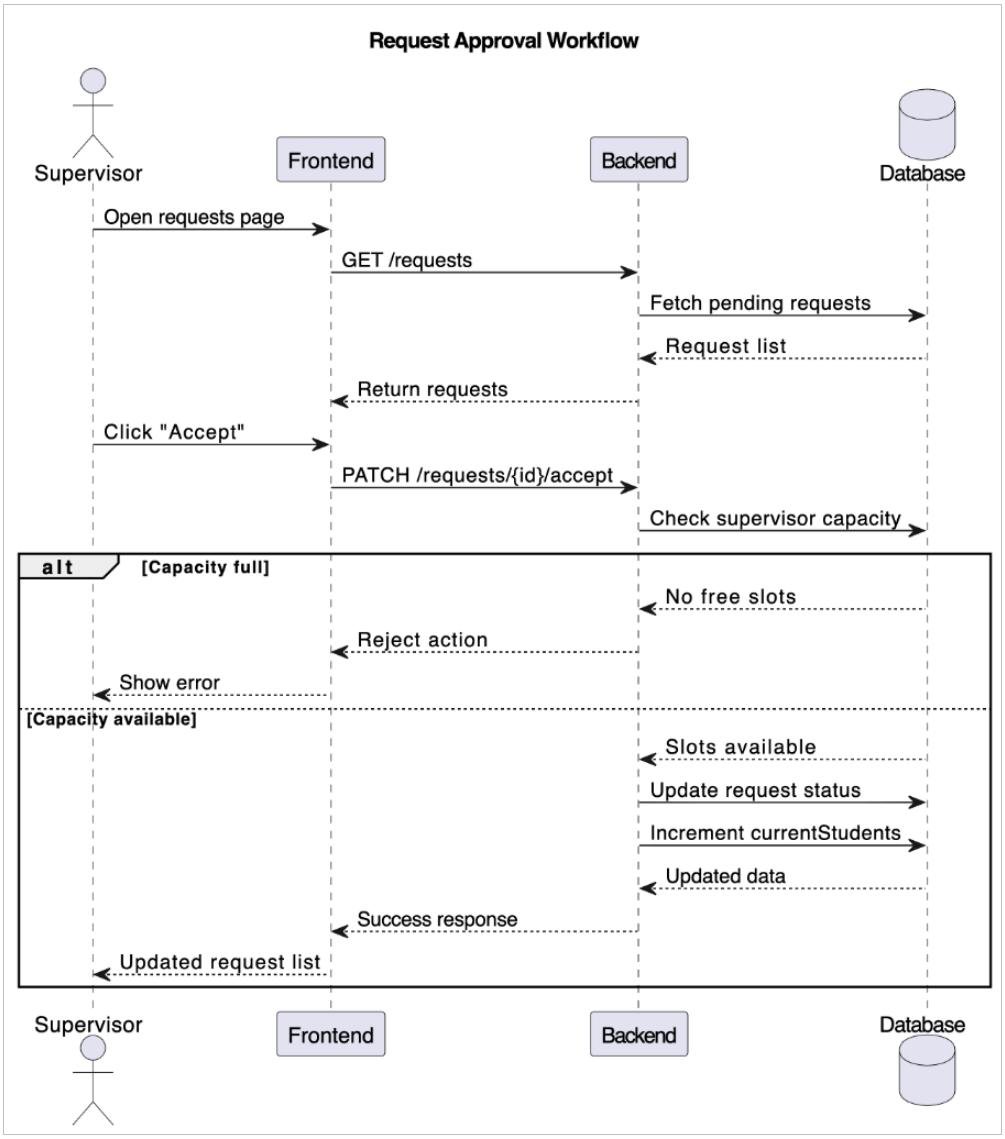

Figure 2.6: Sequence diagram: Request approval workflow

The request approval workflow (Figure 2.6) begins when the **Supervisor** navigates to their dashboard in the **React SPA** and views the list of pending requests. The dashboard's `RequestsTable` component fetches the supervisor's requests on mount via `GET /api/requests/my`, which returns all requests where the authenticated user is the target supervisor. The supervisor then clicks the "Approve" button on a specific request row. The `onClick` handler calls

`api.approveRequest(requestId)`, dispatching `PATCH /api/requests/:id/approve` with the supervisor's JWT in the `Authorization` header to the **Nginx Proxy**.

The **NestJS** `RequestsController` first passes the request through the global `JwtAuthGuard` to validate the token. It then applies the custom `RolesGuard` with the `@Roles(Role.SUPERVISOR)` decorator, which reads the `role` claim from the decoded JWT and rejects the call with `403 Forbidden` if the caller is not a Supervisor. Upon passing both guards, the controller invokes `RequestsService.approveRequest(id, supervisorId)`. The service executes a **Prisma** `$transaction` block that atomically performs two operations against **PostgreSQL**: (1) updating the target `Request` record's `status` field from `PENDING` to `APPROVED`, and (2) decrementing the supervisor's `remainingCapacity` by one. The use of a database transaction ensures that if either operation fails (e.g., due to a concurrent update), neither change is committed, preserving data consistency. PostgreSQL confirms both writes, Prisma returns the updated request object, and the API responds with `HTTP 200 OK`. The SPA re-fetches the requests list and updates the row status badge from "Pending" to "Approved". The capacity badge on the supervisor's public profile is also decremented the next time any user loads the directory.

2.4.3 Activity Diagrams

Student Workflow Activity Diagram

The student workflow activity diagram (Figure 2.7) models the complete lifecycle of a student user's interaction with SupervisorMatch. The flow begins at the **Initial Node** with the student visiting the application. The first decision point asks whether the student already has an account: if not, the flow proceeds to the **Register** activity, in which the student provides their full name, email, password, and selects the `STUDENT` role; on successful registration, they are automatically redirected to the login page. If the student already has an account, or immediately after registration, the flow enters the **Login** activity where credentials are validated and a JWT is returned and stored in the browser.

After authentication, the student enters the **Browse Supervisor Directory** activity, which renders all supervisor profile cards. A fork node then presents two concurrent options: the student may apply a **Search / Filter** (by entering a keyword or filtering by department), which loops back to the directory with filtered results, or proceed directly. At any point the student may **Select a Supervisor Profile** to view detailed information. The next decision node evaluates **Capacity Available?**: if `remainingCapacity = 0`, the Submit Request button is disabled and the student is directed back to browse other supervisors. If capacity is available, the student may **Submit a Supervision Request** by providing an optional message; the system records the request with `status: PENDING`. From their **Student Dashboard**, the student may **Track Request Status** at any time, observing the transition from Pending to either Approved or Rejected as the supervisor acts. The workflow terminates at the **Final Node** when the student receives an Approved request and proceeds to work with their chosen supervisor.

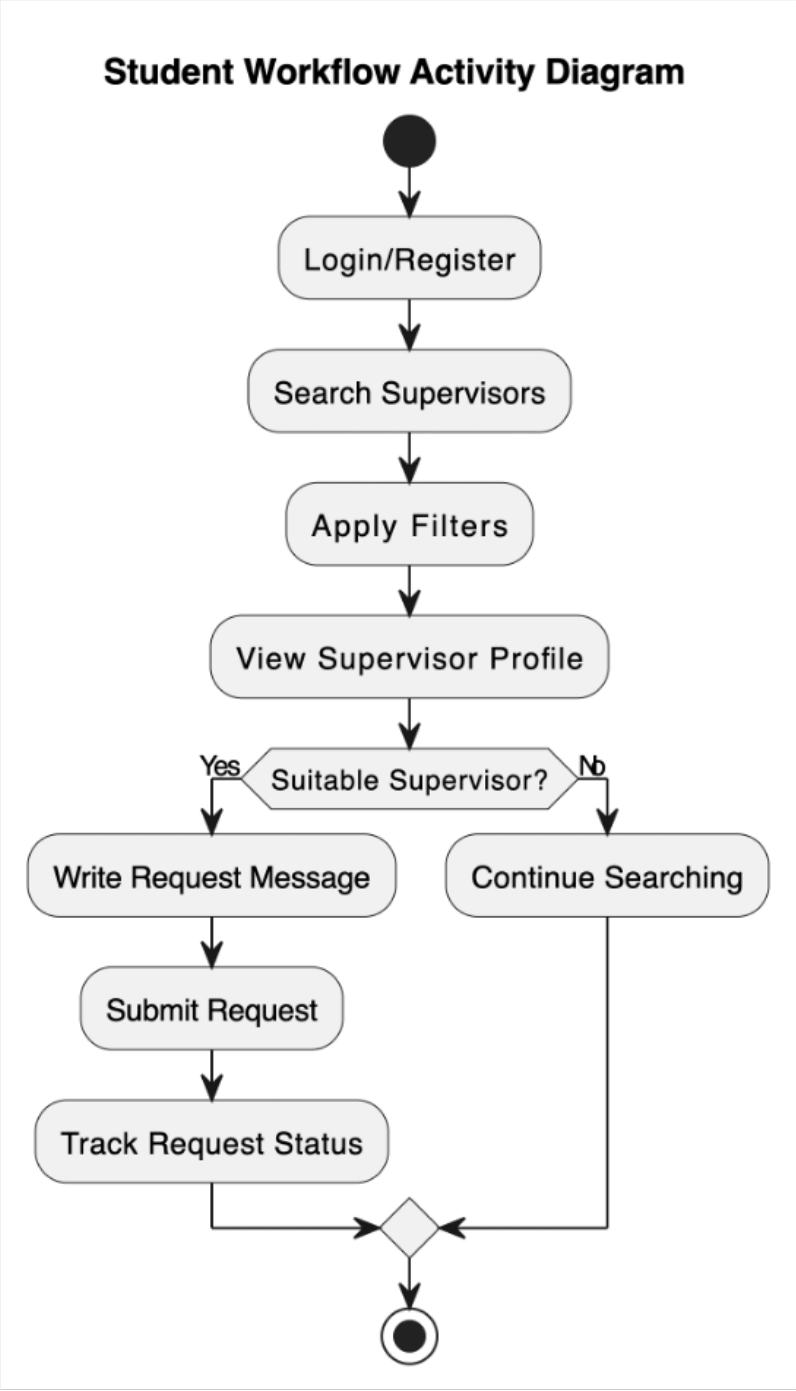

Figure 2.7: Activity diagram: Student workflow

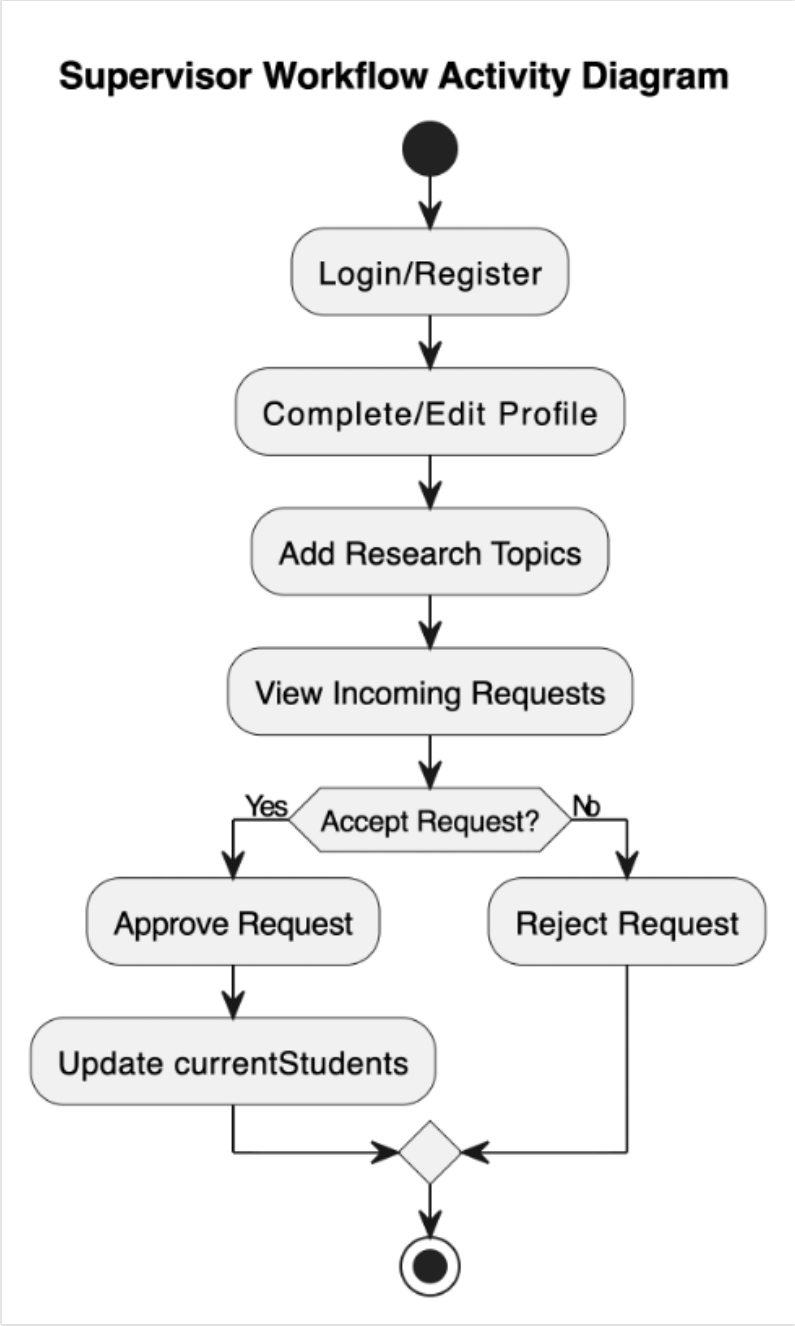

Figure 2.8: Activity diagram: Supervisor workflow

Supervisor Workflow Activity Diagram

The supervisor workflow activity diagram (Figure 2.8) models the full sequence of activities available to a Supervisor-role user. Registration follows the same initial path as for students, with the distinction that the user selects `SUPERVISOR` as their role during the **Register** activity, causing the system to also create a linked `Supervisor` profile record with default capacity values. After **Login**, the supervisor arrives at their dedicated **Supervisor Dashboard**.

The dashboard presents three parallel swim-lanes of activity, modelled as concurrent fork branches: **Profile Management**, **Topic Management**, and **Request Management**. In the Profile Management lane, the supervisor may at any time **Edit Profile** (updating bio, department, and academic title) and **Set Capacity Limit** (adjusting the total number of students they are willing to supervise); a guard condition ensures that capacity cannot be set below the number of already-approved requests. In the Topic Management lane, the supervisor may **Add Research Topic** (providing a title and description), **Edit Topic**, or **Delete Topic**; a deletion guard prevents removal of a topic that is referenced by a pending or approved request. In the Request Management lane, the supervisor **Views Incoming Requests** and then reaches a decision node: for each request, the supervisor may either **Approve Request** (triggering the atomic status update and capacity decrement) or **Reject Request** (updating status to Rejected without affecting capacity). After acting on all pending requests, the supervisor may return to any of the three lanes or **Logout**, terminating the workflow at the **Final Node**.

Administrator Workflow Activity Diagram

The administrator workflow activity diagram (Figure 2.9) documents the planned interaction model for the future `ADMIN` role, which is not active in the current release but is accounted for in the data model's `Role` enum. After **Login with Admin Credentials**, the administrator arrives at an **Admin Control Panel** distinct from both the student and supervisor dashboards. The first available activity is **View All Users**, which presents a paginated table of all registered accounts, their roles, registration dates, and account status. From this view, the administrator may **Promote / Demote Role** (changing a user between Student and Supervisor), **Deactivate Account** (soft-deleting a user without removing their historical request data), or **Reset Password** (generating a temporary credential).

The second major branch is **Monitor Supervision Workload**, which aggregates approved request counts per supervisor and visualises department-level workload distribution. A decision node evaluates whether any supervisor's approved count exceeds a configurable **Overload Threshold**: if so, the system flags that supervisor for administrative review. The third branch, **Export Data**, allows the administrator to download a CSV or PDF report of all requests for a given academic year, supporting end-of-semester auditing by the School of Software Engineering. All three branches are parallel and may be accessed in any order; the administrator terminates the session at the **Final Node** by logging out.

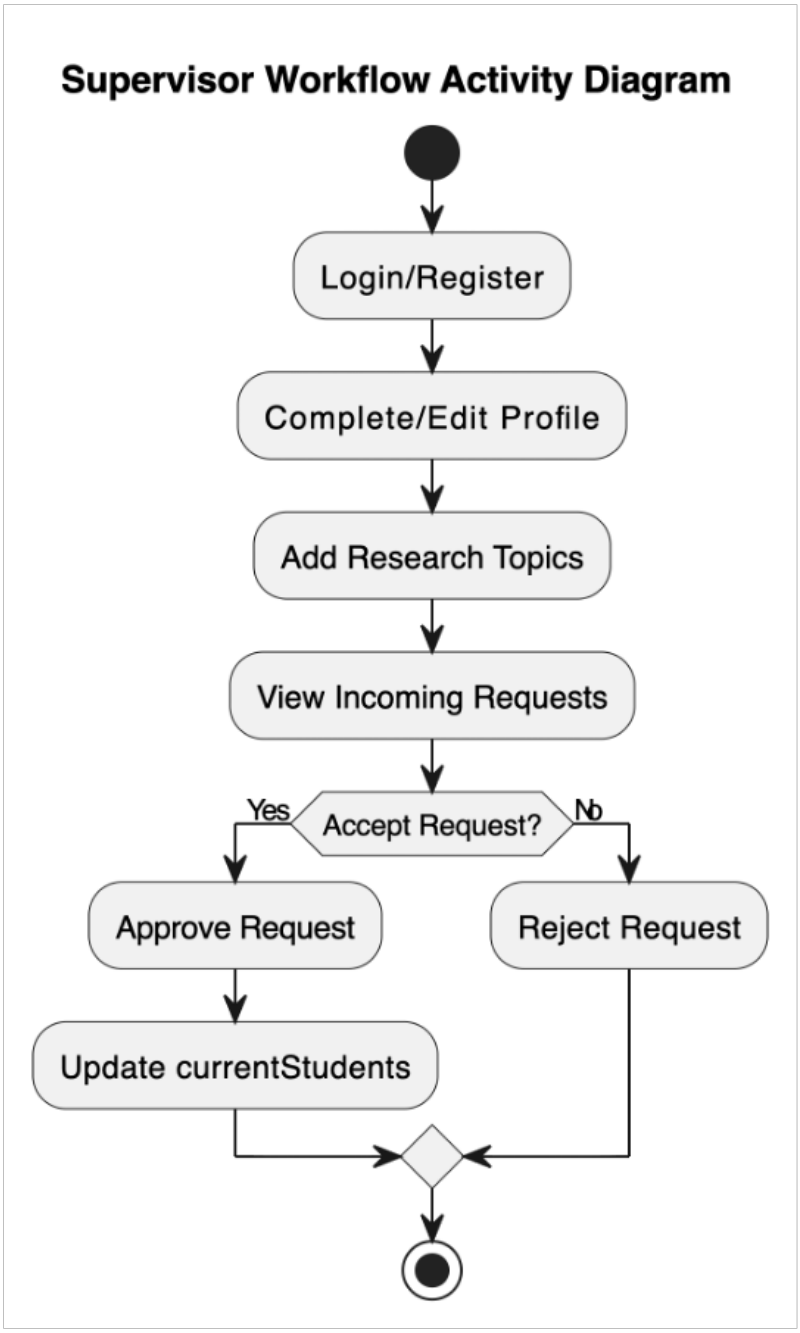

Figure 2.9: Activity diagram: Administrator workflow (planned for future release)

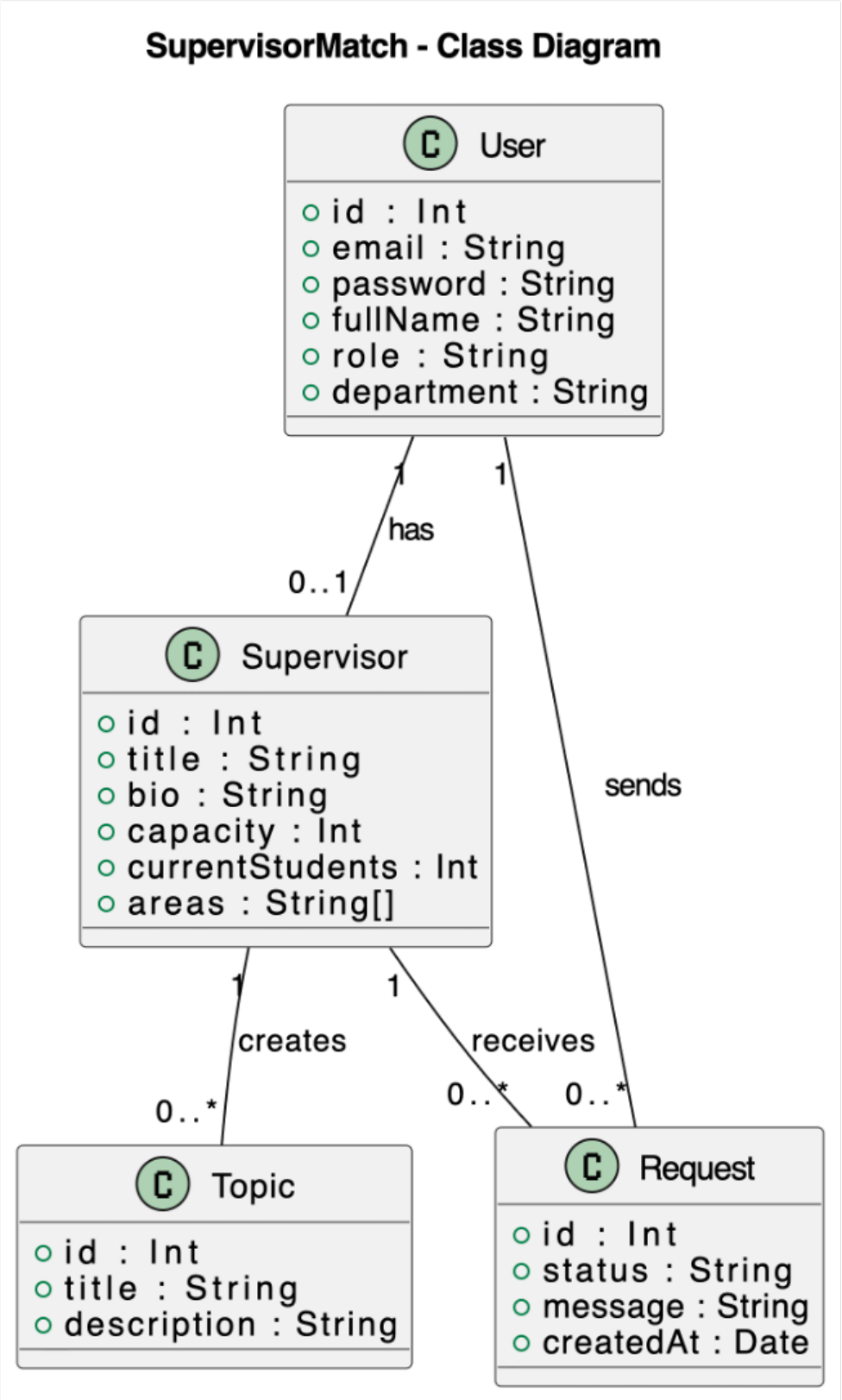

Figure 2.10: UML class diagram for the SupervisorMatch data model

2.4.4 Class Diagram

The UML class diagram (Figure 2.10) represents the four principal domain entities of SupervisorMatch and the associations between them. The **User** class is the root authentication entity, containing the attributes `id: String` (CUID primary key), `email: String` (unique), `password: String` (bcrypt hash), `name: String`, `role: Role` (enumeration with values `STUDENT` and `SUPERVISOR`), and `createdAt: DateTime`. User exposes the operations `register()` and `login():JWT`.

The **Supervisor** class extends the User concept via a *one-to-one* association (a User with `role = SUPERVISOR` has exactly one associated Supervisor record). Supervisor adds the attributes `bio: String?`, `department: String?`, `capacity: Int` (the total declared limit), and `remainingCapacity: Int` (the live counter, initialised equal to `capacity` and decremented on each approval). It exposes the operations `updateProfile()`, `setCapacity()`, and `getRemainingCapacity():Int`.

The **Topic** class represents a research area published by a supervisor. It has attributes `id: String`, `title: String`, `description: String?`, and `supervisorId: String` (foreign key). Topics are related to Supervisor by a *one-to-many* composition: one Supervisor owns zero or more Topics; deleting a Supervisor cascades to their Topics. Topic exposes `create()`, `update()`, and `delete()` operations, the latter being guarded against deletion when a related Request is active.

The **Request** class represents a student's formal supervision application. Its attributes are `id: String`, `studentId: String` (FK to User), `supervisorId: String` (FK to Supervisor), `topicId: String?` (optional FK to Topic), `message: String?`, `status: RequestStatus` (enumeration: `PENDING`, `APPROVED`, `REJECTED`), and `createdAt: DateTime`. Request is associated with User (as student) by a *many-to-one* relationship, and with Supervisor by a *many-to-one* relationship, making the overall Student–Supervisor relationship a *many-to-many* mediated through the Request entity. The class exposes `submit()`, `approve()`, and `reject()` operations, with `approve()` triggering the `Supervisor.remainingCapacity` decrement as a side effect modelled by a dependency arrow.

2.5 Technology Stack Selection

Frontend: React with Vite and Tailwind CSS

React was chosen as the frontend library on the basis of its component model, which naturally maps to the reusable UI elements required by SupervisorMatch (supervisor cards, topic lists, request status badges). The Vite build tool was selected over Create React App for its significantly faster hot module replacement and optimised production build pipeline. Tailwind CSS provides a utility-first styling approach that accelerates UI development without requiring a separate design system, and its responsive breakpoint utilities satisfy the mobile-compatibility non-functional requirement with minimal additional code.

Backend: NestJS with TypeScript

NestJS was selected as the backend framework because its opinionated module architecture enforces a clean separation between application domains (Auth, Supervisors, Topics, Requests), reducing the risk of tight coupling as the codebase grows. Its native TypeScript support enables compile-time type checking across DTO validation, service interfaces, and Prisma-generated types, significantly reducing runtime type errors. The framework's built-in support for guards, interceptors, and pipes provides a clean, declarative mechanism for implementing JWT validation and input validation without boilerplate.

Database: PostgreSQL with Prisma ORM

PostgreSQL was chosen as the relational database for its robustness, ACID compliance, and support for complex queries involving joins across the User, Supervisor, Topic, and Request entities. Prisma ORM was selected to provide type-safe database access: the `prisma generate` command produces a fully typed client from the schema definition, allowing TypeScript to surface query errors at compile time. Prisma Migrate provides a declarative, version-controlled migration system that simplifies schema evolution throughout the development lifecycle. A performance benchmarking study by Mustakim et al. [20] finds that while raw SQL outperforms Prisma on execution speed under extreme load, the developer productivity gains from Prisma's type-safe query API and automatic migration tooling justify its use in applications where developer experience and maintainability are primary concerns, as is the case in SupervisorMatch. The Prisma data guide [21] further identifies Prisma as the only ORM in the Node.js ecosystem that provides full type safety for partial queries and relation traversals, reducing the risk of runtime type errors that are common in traditional ORMs such as Sequelize or TypeORM.

Chapter 3. Implementation

3.1 Backend Implementation

The SupervisorMatch backend is structured as a NestJS application decomposed into four primary feature modules: `AuthModule`, `SupervisorsModule`, `TopicsModule`, and `RequestsModule`. A shared `PrismaModule` provides a singleton `PrismaService` instance that is injected into each feature module's service class, ensuring a single database connection pool is shared across the application.

Authentication Module

Authentication is implemented using the `@nestjs/passport` library with a `LocalStrategy` for credential validation and a `JwtStrategy` for token verification. Upon successful login, the `AuthService` issues a signed JWT containing the user's `id`, `email`, and `role` fields, with a configurable expiry period defined via environment variable. The `JwtAuthGuard` is applied globally to all routes, with the `@Public()` decorator used to exempt the `/auth/register` and `/auth/login` endpoints. Password hashing uses the bcrypt library with a salt factor of ten rounds.

```
async login(user: User) {
  const payload = {
    sub: user.id,
    email: user.email,
    role: user.role,
  };
  return {
    access_token: this.jwtService.sign(payload),
  };
}
```

Listing 3.1: JWT payload generation in AuthService

Requests Module

The `RequestsModule` implements the core business logic of the application. When a student submits a request (POST `/requests`), the service checks that the target supervisor's `remainingCapacity` is greater than zero before creating the request record. If capacity is exhausted, a `BadRequestException` is thrown. When a supervisor approves a request, the service updates the request status to `APPROVED` and atomically decrements the supervisor's `remainingCapacity` within a Prisma transaction, preventing race conditions under concurrent load.

```
async approveRequest(requestId: string, supervisorId: string) {
  return this.prisma.$transaction(async (tx) => {
    const request = await tx.request.update({
      where: { id: requestId },
      data: { status: 'APPROVED' },
    });
    await tx.supervisor.update({
      where: { id: supervisorId },
      data: { remainingCapacity: { decrement: 1 } },
    });
    return request;
  });
}
```

Listing 3.2: Capacity check and transactional approval in RequestsService

3.2 Frontend Implementation

The React frontend is organised around a top-level `App.tsx` component that uses React Router v6 for client-side navigation. Protected routes are implemented via a `PrivateRoute` component that reads the JWT from `localStorage`, decodes the role claim, and redirects unauthenticated or unauthorised users to the login page.

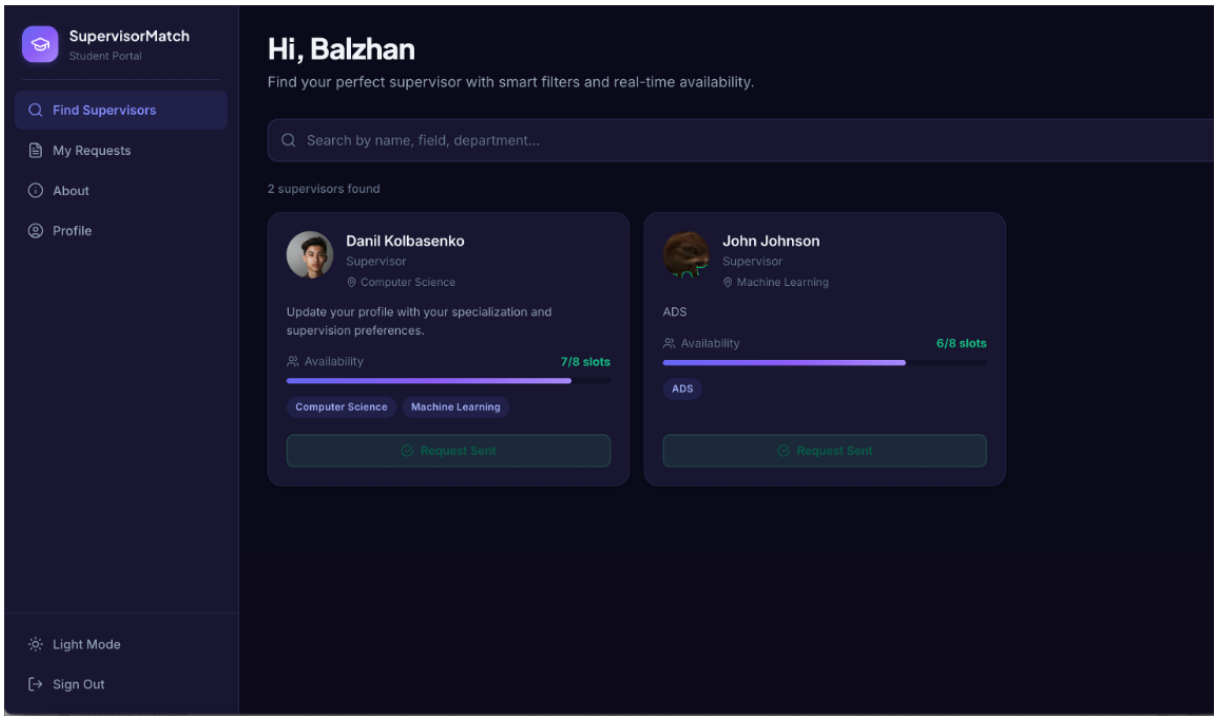

Figure 3.1: SupervisorMatch — Supervisor Directory user interface

The application exposes four primary views: a public Supervisor Directory (accessible without authentication), a Supervisor Profile Detail page, a Student Dashboard displaying the authen-

ticated student's submitted requests and their current status, and a Supervisor Dashboard showing incoming requests with approve/reject controls. All API calls are centralised in a `services/api.ts` module using Axios with an interceptor that attaches the Bearer token to every outbound request.

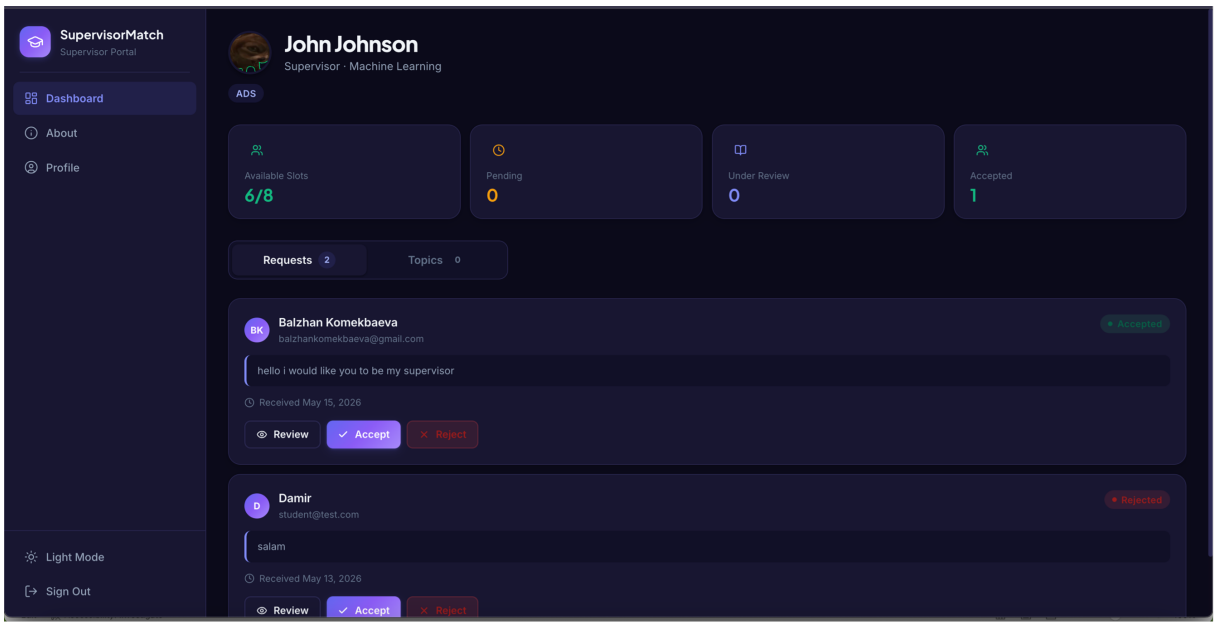

Figure 3.2: SupervisorMatch — Supervisor Dashboard user interface

3.3 Database Design

The database schema is defined in a single `schema.prisma` file and managed through Prisma Migrate. The schema defines four primary models: `User`, `Supervisor`, `Topic`, and `Request`. The `User` model stores authentication credentials and the role discriminator. The `Supervisor` model extends `User` with profile information and capacity fields, linked via a one-to-one relation. `Topic` records are owned by a `Supervisor` via a one-to-many relation. `Request` records establish a many-to-many relationship between `User` (student) and `Supervisor`, enriched with a status enum and an optional message field.

```
model User {
  id          String      @id @default(cuid())
  email       String      @unique
  password    String
  role        Role        @default(STUDENT)
  name        String
  supervisor  Supervisor?
  requests    Request[]
  createdAt   DateTime    @default(now())
}

model Supervisor {
```

```
  id                String    @id @default(cuid())
  userId            String    @unique
  user              User      @relation(fields: [userId],
     references: [id])
  bio               String?
  department        String?
  capacity          Int       @default(5)
  remainingCapacity Int       @default(5)
  topics            Topic[]
  requests          Request[]
}

model Topic {
  id           String     @id @default(cuid())
  title        String
  description  String?
  supervisorId String
  supervisor   Supervisor @relation(fields: [supervisorId],
     references: [id])
  requests     Request[]
}

model Request {
  id           String        @id @default(cuid())
  studentId    String
  student      User          @relation(fields: [studentId],
     references: [id])
  supervisorId String
  supervisor   Supervisor    @relation(fields: [supervisorId],
     references: [id])
  topicId      String?
  topic        Topic?        @relation(fields: [topicId],
     references: [id])
  message      String?
  status       RequestStatus @default(PENDING)
  createdAt    DateTime      @default(now())
}

enum Role          { STUDENT SUPERVISOR }
enum RequestStatus { PENDING APPROVED REJECTED }
```

Listing 3.3: Prisma schema definitions for core models

3.4 Docker Containerisation

The application is containerised using Docker and orchestrated with Docker Compose. Three services are defined in the `docker-compose.yml` file: `db` (PostgreSQL 15), `api` (NestJS application), and `frontend` (Nginx serving the Vite production build and reverse-proxying API requests). Thokala [23] establishes that reproducible container builds via Docker ensure environment parity between development and production, eliminating the category of "works on my machine" defects; this benefit is of particular relevance in a two-developer academic project context where local environment divergence is a common source of integration failures.

The `api` service depends on `db` and waits for the database to be healthy via a Docker health check before executing `prisma migrate deploy` and `node dist/main.js`. Environment variables (database URL, JWT secret, port) are injected via a `.env` file excluded from version control. The Nginx configuration serves the React SPA for all non-API routes (enabling client-side routing with React Router) and forwards requests to `/api/` to the `api` container on port 3000.

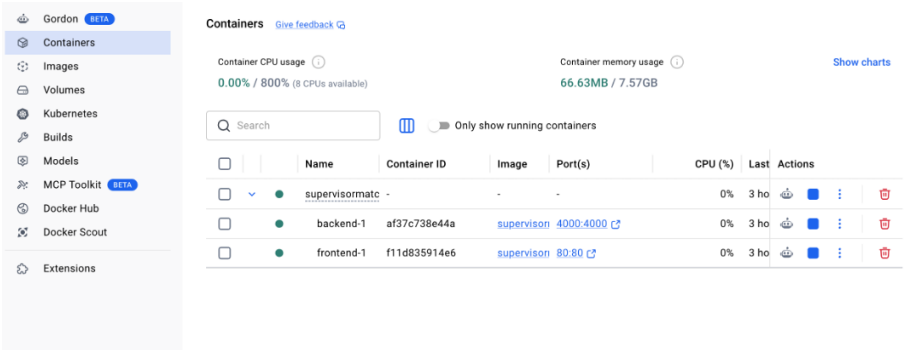

Figure 3.3: Docker deployment architecture for SupervisorMatch

Conclusion

Project Summary

This project has documented the design, development, and deployment of SupervisorMatch, a full-stack web application that digitalises the academic supervisor discovery and request management process at Astana IT University. The application successfully delivers the core functional requirements identified through literature review, comparative analysis, and empirical student survey: a searchable supervisor directory with real-time capacity visibility, a structured online supervision request workflow, role-based access control for Student and Supervisor roles, and a fully containerised deployment architecture.

Contributions and Innovation

The primary technical contribution of this project is the integration of real-time capacity management into the supervision request lifecycle—a feature absent from all comparable systems identified in the literature review. By automatically enforcing the supervisor's self-declared capacity limit and reflecting remaining availability on every supervisor profile card, SupervisorMatch eliminates the primary source of student frustration identified by the survey: contacting supervisors who are already fully committed. The use of Prisma transactions to ensure atomic capacity updates under concurrent request submission represents a deliberate engineering choice to maintain data integrity at scale.

Development Approach Effectiveness

The Agile/Scrum methodology proved effective for a two-person development team within a semester-length timeline. The two-week Sprint cadence provided sufficient granularity to detect integration problems early—particularly at the JWT authentication boundary between frontend and backend—and to reprioritise backlog items in response to testing feedback. The MoSCoW prioritisation method ensured that the most critical functional requirements (authentication, supervisor discovery, request submission, approval workflow) were completed and tested before effort was invested in secondary features.

Future Enhancement Possibilities

Several directions for future development are identified. First, the integration of a machine learning recommender system that suggests supervisors to students based on their academic interests, stated research goals, and historical matching outcomes represents the most impactful potential enhancement. Yera et al. [7], in a systematic review of 56 recommender system studies in higher

education published between 2011 and 2023, find that hybrid strategies combining collaborative filtering with content-based methods consistently outperform single-approach systems, and that matrix factorisation over research keyword embeddings is the most promising technique for supervisor–student matching. Braescu and Davidescu [6] specifically propose cosine similarity over topic description vectors as a practical starting point for such a recommender in an educational web application context, providing a directly applicable blueprint for SupervisorMatch's next development iteration.

Second, integration with the university's existing identity provider via OAuth 2.0 or SAML would eliminate the need for a separate registration flow and would associate SupervisorMatch accounts with verified institutional identities, increasing trust and data accuracy. Third, an email and in-app notification system would reduce response latency by alerting supervisors to new requests and informing students of status changes in real time. Finally, an administrative dashboard providing system-wide visibility into supervision load distribution across departments would support institutional planning and workload equity monitoring.

Appendix A. Appendix A: API Endpoint Reference

A.1 SupervisorMatch REST API Reference

All endpoints are served at the base URL configured via the `PORT` environment variable (default: `http://localhost:4000`). Request and response bodies use JSON encoding unless stated otherwise. Authenticated endpoints require a valid JWT access token in the `Authorization` header using the Bearer scheme.

A.1.1 Authentication

`POST /auth/register`

Creates a new user account. If the role is `supervisor`, the system additionally generates a linked Supervisor profile record with default capacity values.

Request body

Response (200)

```
{
  "ok": true,
  "user": {
    "id": "uuid",
    "fullName": "Demo Student",
    "email": "student@example.com",
    "role": "student",
    "department": "Computer Science",
    "createdAt": "2026-05-10T12:00:00.000Z"
  }
}
```

Error codes: 400 (invalid payload), 409 (email already registered).

`POST /auth/login`

Authenticates a user by email and password. On success, the server returns a short-lived JWT access token in the response body and sets a long-lived refresh token as an `httpOnly` cookie.

Request body

Response (200)

```
{
  "ok": true,
  "accessToken": "<JWT>",
  "user": {
    "id": "...",
    "fullName": "...",
    "email": "...",
    "role": "student"
  }
}
```

The `refreshToken` cookie is set with the following attributes: `httpOnly`, `sameSite: lax`, `maxAge: 7 days`. In production mode the `Secure` flag is also enabled.

Error codes: 400 (invalid payload), 401 (wrong credentials).

`POST /auth/refresh`

Issues a new access token using the refresh token stored in the `refreshToken` cookie.

Response (200)

```
{ "ok": true, "accessToken": "<JWT>" }
```

Error codes: 401 (missing, expired, or revoked refresh token).

`POST /auth/logout`

Auth required: yes.

Invalidates the current refresh token by clearing its hash from the database and removing the `refreshToken` cookie from the client.

Response (200)

```
{ "ok": true }
```

A.1.2 User Profile

`GET /users/me`

Auth required: yes.

Returns the profile of the currently authenticated user.

Response (200)

```
{
  "id": "uuid",
  "fullName": "Alex Carter",
  "email": "alex@example.com",
  "role": "student",
  "department": "Computer Science",
  "groupName": "SE-2201",
  "phone": "+77001234567",
  "studyLevel": "Bachelor",
  "interests": "AI, Machine Learning",
  "bio": "Final-year student...",
  "avatar": "...",
  "createdAt": "2026-01-15T09:00:00.000Z"
}
```

`PATCH /users/me`

Auth required: yes.

Updates profile fields of the authenticated user.

Request body

A.1.3 File Upload

`POST /upload`

Auth required: yes.

Accepts a file via `multipart/form-data`, converts it to a Base64-encoded data URL, and returns it in the response.

Response (200)

```
{ "url": "..." }
```

Error codes: 400 (no file provided).

A.1.4 Supervisors

`GET /supervisors`

Public endpoint. Returns a filtered list of all registered supervisors along with their associated topics.

Supported query parameters

Field	Type	Required	Constraints
fullName	string	yes	min. 2 characters
email	string	yes	valid email format
password	string	yes	min. 6 characters
role	string	yes	`"student"` or `"supervisor"`
department	string	yes	non-empty
groupName	string	no	defaults to `""`

Table A.1: Register endpoint request body

Field	Type	Required
email	string	yes
password	string	yes

Table A.2: Login endpoint request body

Field	Type	Description
fullName	string	min. 2 characters
department	string	non-empty
groupName	string	student group
phone	string	contact number
studyLevel	string	e.g. Bachelor, Master
interests	string	comma-separated list
bio	string	biography text
title	string	academic title
areas	string[]	research areas
avatar	string	Base64 data URL or image URL

Table A.3: Profile update request body

Parameter	Type	Description
keyword	string	Case-insensitive substring search across supervisor fields
department	string	Exact match on department name
area	string	Exact match against supervisor research areas
availableOnly	boolean	Excludes supervisors whose capacity is full

Table A.4: Supervisor filtering parameters

`GET /supervisors/:id`

Returns a single supervisor record identified by ID.

Error codes: 404 (supervisor not found).

`PATCH /supervisors/:id/profile`

Auth required: yes (supervisor role only).

Updates the supervisor's public-facing profile.

Request body

A.1.5 Topics

`POST /supervisors/:id/topics`

Auth required: yes (supervisor role only).

Creates a new research topic associated with the supervisor.

Request body

A.1.6 Supervision Requests

`POST /requests`

Auth required: yes (student role only).

Submits a formal supervision request to a supervisor.

Request body

Response (200)

```
{
  "id": "...",
  "studentUserId": "...",
  "studentName": "...",
  "studentEmail": "...",
  "supervisorId": "...",
  "message": "...",
  "status": "pending"
}
```

`PATCH /requests/:id/status`

Auth required: yes (supervisor role only).

Updates the status of a supervision request.

Allowed status values

Request body

```
{ "status": "accepted" }
```

A.1.7 Administrator API

All endpoints under the `/admin` prefix require a separate administrator JWT.

`POST /admin/auth/login`

Authenticates the system administrator.

Request body

Response (200)

```
{
  "token": "<admin JWT>",
  "user": {
    "id": "admin-001",
    "name": "Administrator",
    "email": "admin@supervisormatch.local",
    "role": "super_admin"
  },
  "expiresIn": 86400
}
```

`GET /admin/dashboard/stats`

Auth required: yes (admin).

Returns aggregated statistics for the dashboard overview.

`GET /admin/students`

Auth required: yes (admin).

Returns a paginated, filterable list of all registered students.

Supported query parameters

Field	Type	Required	Constraints
name	string	yes	min. 2 characters
department	string	yes	non-empty
areas	string[]	yes	at least one entry
title	string	no	defaults to “Supervisor”
phone	string	no	–
bio	string	no	–
avatar	string	no	retains previous value if omitted

Table A.5: Supervisor profile update request

Field	Type	Required
title	string	yes
area	string	yes
description	string	yes

Table A.6: Topic creation request

Field	Type	Required	Description
supervisorId	string	yes	target supervisor ID
message	string	no	motivation text

Table A.7: Supervision request payload

Status	Description
pending	Initial state after submission
under review	Supervisor has begun evaluation
accepted	Supervisor approved the request
rejected	Supervisor declined the request

Table A.8: Supervision request statuses

Field	Type	Required
login (or email)	string	yes
password	string	yes

Table A.9: Administrator login request

Parameter	Type	Default	Description
page	integer	1	page number
pageSize	integer	10	results per page
search	string	–	substring search
status	string	–	active, pending, inactive
sortBy	string	fullName	sorting field
sortDir	string	asc	asc or desc

Table A.10: Student list query parameters

`GET /admin/analytics`

Auth required: yes (admin).

Returns a comprehensive analytics payload covering application statistics, department distributions, supervisor load, approval rate, and request summaries.

A.1.8 Error Response Format

All error responses follow a consistent JSON structure:

```
{
  "message": "Human-readable error description"
}
```

Validation errors may additionally include a `details` field containing structured validation information.

A.1.9 Authentication Flow Summary

The system implements a dual-token authentication strategy:

1. **Access Token** — a short-lived JWT (TTL: 1 hour) containing the user ID, role, and email claims, signed with the `JWT_ACCESS_SECRET`. Sent in the `Authorization: Bearer <token>` header.
2. **Refresh Token** — a long-lived JWT (TTL: 7 days) signed with the `JWT_REFRESH_SECRET`. Stored as an `httpOnly` cookie and persisted in the database as a SHA-256 hash.

Password hashing uses SHA-256 with a configurable salt defined in the `HASH_SALT` environment variable.

Appendix B. Appendix B: Survey Questionnaire

The following questionnaire was distributed to AITU students in January 2026 via Google Forms to collect data on their supervisor selection experience.

1. What is your level of study? [Bachelor / Master / PhD]
2. Are you currently required to choose an academic supervisor? [Yes / No]
3. How did you search for an academic supervisor? [Multiple choice]
4. How easy was it to find information about supervisors' research interests? [1–5 scale]
5. Was information about supervisors' current supervision workload available? [Yes / Partially / No]
6. How many supervisors did you contact before receiving approval? [One / Two / Three or more / Not yet approved]
7. What difficulties did you experience? [Multiple choice]
8. How long did the supervisor selection process take? [<1 week / 1–2 weeks / >2 weeks]
9. How transparent do you find the current process? [1–5 scale]
10. Would you use a centralised web platform to search for supervisors? [Yes / No / Not sure]
11. Which platform features would be most useful? [Multiple choice]
12. How important is institutional authorisation? [1–5 scale]
13. What is your biggest frustration when choosing a supervisor? [Open text]

Bibliography

[1] Johnson, W. B. (2015). *On Being a Mentor: A Guide for Higher Education Faculty* (2nd ed.). Routledge.

[2] Gale, D., & Shapley, L. S. (1962). College admissions and the stability of marriage. *The American Mathematical Monthly*, 69(1), 9–15. https://doi.org/10.1080/00029890.1962.11989827

[3] Park, S., & Lee, J. (2018). Design and evaluation of an online supervisor recommendation system for graduate students. *Computers & Education*, 127, 139–151. https://doi.org/10.1016/j.compedu.2018.08.011

[4] Ismail, M. H., Razak, T. R., Hashim, M. A., & Ibrahim, A. F. (2019). A simple recommender engine for matching final-year project student with supervisor. *arXiv preprint arXiv:1908.03475*. https://arxiv.org/abs/1908.03475

[5] Shakerian, S., Noori, S., Hiedarpoor, P., Shams, L., & Hosseinzadeh, M. (2020). Developing a web-based learning management system (LMS) for master's thesis process. *Journal of Medical Education*, 19(4), e110696. https://doi.org/10.5812/jme.110696

[6] Braescu, A. I., & Davidescu, A. (2024). Revolutionizing higher education: An in-depth exploration of advanced recommendation systems for project selection and supervision. In *Advances in Intelligent Systems and Computing*. Springer. https://doi.org/10.1007/978-3-032-11013-8_1

[7] Yera, A., Alzahrani, A., & Martinez, L. (2024). Fifteen years of recommender systems research in higher education: Current trends and future direction. *Applied Artificial Intelligence*, 37(1), article 2175106. https://doi.org/10.1080/08839514.2023.2175106

[8] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation, University of California, Irvine). https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[9] Bogner, J., Fritzsch, J., Wagner, S., & Zimmermann, A. (2023). Microservice API evolution in practice: A study on strategies and challenges. *arXiv preprint arXiv:2311.08175*. https://arxiv.org/abs/2311.08175

[10] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*: RFC 7519. Internet Engineering Task Force. https://tools.ietf.org/html/rfc7519

[11] Ferraiolo, D. F., & Kuhn, D. R. (1992). Role-based access controls. In *Proceedings of the 15th National Computer Security Conference*, pp. 554–563. NIST.

[12] Ferraiolo, D., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security*, 4(3), 224–274. https://doi.org/10.1145/501978.501980

[13] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *IEEE Computer*, 29(2), 38–47. https://doi.org/10.1109/2.485845

[14] Bhatti, R., Bertino, E., & Ghafoor, A. (2003). Role-based access control on the web. *ACM Transactions on Information and System Security*, 6(4), 432–465. https://doi.org/10.1145/950191.950193

[15] Shin, M. E., Ahn, G.-J., & Cho, S.-H. (2012). Roles-based access control modeling and testing for web applications. In *Proceedings of the IEEE 36th Annual Computer Software and Applications Conference*, pp. 450–455. IEEE. https://doi.org/10.1109/COMPSAC.2012.81

[16] Jonathan, R., & Suprihadi. (2023). Development of front-end web applications utilizing single page application framework and React.js library. *International Journal of Software Engineering and Computer Science (IJSECS)*, 3(3), 201–213. https://doi.org/10.35870/ijsecs.v3i3.1943

[17] Piastou, M. (2023). Comprehensive performance and scalability assessment of front-end frameworks: React, Angular, and Vue.js. *World Journal of Advanced Engineering Technology and Sciences*, 9(2), 366–376. https://doi.org/10.30574/wjaets.2023.9.2.0153

[18] Priyanshi, E. R., & Vashishtha, S. (2023). Angular vs. React: A comparative study for single page application development. *International Journal of Computer Science Publications (IJCSP)*, 13(1), 80–91.

[19] Wang, L. (2024). Practical approaches to developing high-performance web applications based on React. *Frontiers in Science and Engineering*, 4(1), 112–120. https://doi.org/10.54097/fse.v4i1.22

[20] Mustakim, M., Raharjo, W., & Handayani, P. W. (2025). Optimizing database access strategy: A performance analysis comparison of raw SQL and Prisma ORM. *Procedia Computer Science*, 251, 490–499. https://doi.org/10.1016/j.procs.2025.02.XXX

[21] Prisma. (2022). Evaluating type safety in the top 8 TypeScript ORMs. Prisma Data Guide. https://www.prisma.io/dataguide/database-tools/evaluating-type-safety-in-the-top-8-typescript-orms

[22] Sharma, N. (2021). Containerization of web application using Docker. *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, 9(6), 2401–2408. https://doi.org/10.22214/ijraset.2021.35152

[23] Thokala, V. S. (2021). Utilizing Docker containers for reproducible builds and scalable web application deployments. *International Journal of Current Engineering and Technology*, 11(6), 659–664.

[24] Cherukuri, B. R. (2024). Containerization in cloud computing: Comparing Docker and Kubernetes for scalable web applications. *International Journal of Science and Research Archive (IJSRA)*, 13(1), 1024–1032. https://doi.org/10.30574/ijsra.2024.13.1.2035

[25] Amaral, M., Polo, J., Carrera, D., Mohomed, I., Unuvar, M., & Steinder, M. (2021). Performance analysis of containerization in cloud computing. *Arabian Journal for Science and Engineering*, 46(4), 3205–3224. https://doi.org/10.1007/s13369-020-05237-6

[26] Beck, K., Beedle, M., van Bennekum, A., et al. (2001). *Manifesto for Agile Software Development*. Agile Alliance. https://agilemanifesto.org

[27] Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.

[28] Widiyatmoko, A. T., Nugroho, A., & Wiyanto, W. (2024). Development of web-based student registration information system with rapid application development approach. *Journal of Computer Networks, Architecture and High Performance Computing*, 6(1), 484–491. https://doi.org/10.47709/cnahpc.v6i1.3459